

Vibe Coding: Working Framework

What This Is

You're learning to build things using AI as your developer. You won't write code yourself – you'll describe what you want, receive code, test it, and direct fixes. The AI will confidently produce broken code, lose track of your project, and add complexity you don't need. Your job is to stay in control.

This is a real skill: commissioning and managing technical work from a system you don't fully understand.

The Rules

1. One thing at a time. Each conversation with the AI should produce one specific thing. Not "build my whole project" – one working piece.

Write your session goal before you start:

- ✓ "An HTML page that displays a grid of images with captions"
- ✓ "A button that changes the background to a random colour"
- ✗ "Build my app" / "Make it look good" / "Add some features"

2. Test immediately. Every time the AI gives you code, check it right away:

- Does it display?
- Does the interaction work?
- What happens with unexpected input?
- Does it still look right on a smaller screen?

Never ask for the next feature until the current one works.

3. Save before changing. Before you ask the AI to modify something that works, copy your working version somewhere safe. The AI may break what was working while adding something new. If it does, you can go back.

4. Describe symptoms, not theories. When something breaks:

- ✗ "I think the CSS is wrong"
- ✓ "The button doesn't appear on the page"
- ✗ "The JavaScript isn't working"
- ✓ "When I click Submit, nothing happens and the console shows: TypeError: cannot read property of null"

You don't need to know *why* it's broken. You need to describe *what you see*.

5. Understand before you proceed. If the AI gives you code and you can't describe in your own words what it's supposed to do, ask for an explanation first. You can't debug what you can't describe.

6. Resist unnecessary complexity. If the AI says you need a backend server, a database, an API key, Node.js, or user authentication – ask: "Can we do a simpler version without that?" Often the answer is yes.

7. Know when to start over. If you've gone back and forth 5–6 times on the same problem and it's not improving, the conversation is broken. Start fresh:

1. Copy out the code that works
2. Open a new conversation
3. Give the AI: what you have (paste the code), what you're trying to add, what went wrong

This is not failure. It's the fastest path forward.

Your Most Useful Debugging Tool

Right-click anywhere on your page → **Inspect** → click the **Console** tab.

Red text here is error messages. You don't need to understand them – copy the exact text and paste it to the AI. This solves most "it doesn't work" situations in one step.

Scope: Must Have Now vs. Later

At any point, you should be able to divide your project into two lists:

Must have now (proves the idea works): the core interaction, one clear example of the functionality, something someone can look at and understand.

Add later (nice but not essential): visual polish, multiple variations, edge cases, mobile layout, animations.

Build "must have now" first. Get it working. Then add one thing from "later." Repeat. If you try to build everything at once, you'll finish nothing.

Warning Signs

Stop and reconsider if:

- The AI keeps apologising ("I apologise for the confusion...")
- Fixes undo previous fixes (fix A breaks B, fix B breaks A)
- Explanations get longer and more complicated with each message
- The AI suggests a completely different approach without explaining why the last one failed
- You've been stuck on the same thing for more than 20 minutes

Any of these means: save what works, start a fresh conversation, describe the problem cleanly.

Quick Reference

Situation	What to do
AI gives you code	Can you explain what it does? If not, ask first.
Something doesn't work	Open the browser console. Copy the red text. Paste it to the AI.
AI suggests something complex	"Can we do a simpler version that doesn't need that?"
Same problem for 5+ messages	Save working code. New conversation. Clean description.
Project feels overwhelming	What's the smallest thing that shows the idea? Build that.
You don't understand something	Ask. Never proceed with what you can't describe.

What Success Looks Like

By the end of this workshop, you should be able to:

1. **Calibrate** – recognise what's an afternoon vs. a multi-week project
2. **Recover** – restart productively when stuck, rather than grinding
3. **Evaluate** – assess "does this do what I asked?" without reading every line
4. **Scope** – decide what to build now vs. what to defer
5. **Communicate** – describe what went wrong in terms that get results

You won't be a programmer. You'll be someone who can direct the process and stay in control of it.